Project 3 — Verified Research Agent — Progress & Handoff Summary

Status: M1 (Setup), M2 (Ingestion + semantic search), and M3 (Hybrid retrieval) COMPLETE — committed and pushed to GitHub. **Next: M4 (Generation). Last updated:** June 2026 (after M3) **Purpose of this doc:** a living handoff document for Project 3 — (1) a record of what's been built so far, and (2) a paste-into-a-new-chat context document so work can resume on the next milestone with zero re-explanation. Updated at each milestone completion.

---

0. How to use this doc in a new chat

Paste this whole file at the top of a new chat, plus tell the assistant: *"Continue this project from M4 (the next milestone). Use the established learning format: one concept/lesson at a time — you explain and give me a task, I run it and report back, then we advance."* The assistant should not need to ask setup questions; everything it needs is below. (M1–M3 are done — see §10 for the milestone table and §14 for exactly where to resume.)

---

1. Who I am / project context

- Master's student in Germany, 2 years experience, targeting FAANG + AI-native ML/LLM Engineer roles.
- Specialization: NLP/LLM safety & reliability.
- This is **Project 3** of a 3-project portfolio. The unified story: Project 2 trained a model to reduce hallucinations (prevention); the thesis built a verifier to detect them (detection); Project 3 deploys a production research agent that verifies its own claims using the thesis verifier (deployment), benchmarked and load-tested at serving scale.
- Project 3 is following the **v2 (Production-Scale Edition)** plan: 12 milestones (M1–M12). v2 adds a serving-benchmark / load-test / cost / public-demo block (M9–M12) on top of v1's M1–M8, and makes generation an external vLLM HTTP service from the start.
- The one-line through-line: *"A research agent that measures and reports the grounding of its own answers — turning hallucination rate from an offline eval number into a live production observable."*

Working environment (important — non-standard bits)

- **OS / shell:** Windows, using **CMD** (not PowerShell, not Git Bash).
- **Python env:** conda env `fastapi-env` (this is the Project 3 env; Project 2 uses a different env called `safeenv` — do not mix them up).

- **Editor:** VS Code (`code <path>`). Note: `code <path>` opens a path **relative to the current terminal directory**.
- **Git commit style:** single-line messages only (multi-line double-quoted messages break in Windows CMD).
- **File delivery preference:** long files delivered as downloadable artifacts, not pasted inline.
- **Learning format:** one lesson/concept at a time — assistant explains + assigns a task, I execute and report results, then we advance. I want to understand every piece, not copy-paste blindly.

---

2. What M1 built (the deliverable — historical record)

(M1 is the setup milestone. M2's deliverable is summarized in §14; per-file detail for both M1 and M2 is in §3–§4.)

M1 goal (met): the full storage + API stack runs locally with no GPU. `docker-compose up` brings up Postgres, Qdrant, and Redis; the FastAPI app serves `/health` (200) and `/metrics`; all 7 Postgres tables exist; the Qdrant collection exists; and the generation client passes a round-trip against a local stub.

Concretely, M1 delivered:

- Repo + Python package structure.
- A config layer that swaps local/Docker/LLM-backend addresses via environment variables (no hardcoded hosts).
- FastAPI app with `/health`, `/metrics`, and `/` root.
- Async SQLAlchemy session + a 7-table schema, created for real in a live Postgres.
- A Qdrant collection `research_papers` (384-dim, cosine).
- A Docker Compose stack (Postgres, Qdrant, Redis) running with persistent volumes.
- A swappable generation client with a working **stub mode** (no GPU needed to build the rest of the pipeline).
- Everything committed locally and pushed to GitHub.

---

3. Repository layout (as built in M1)

```
verified-research-agent/                    <- project root
(C:\Users\mekal\verified-research-agent)
├── docker-compose.yml                      <- postgres, qdrant, redis services
+ volumes
├── .env.example                            <- template of env vars (committed;
real .env is gitignored)
```

```
├── .gitignore                          <- ignores __pycache__, *.pyc, .env,
etc.
├── requirements.txt                    <- pinned dependencies
└── backend/
    ├── create_tables.py                <- one-off script: creates the 7
tables in live Postgres
    ├── data/
    |   └── sample_papers.json          <- [M2] 6-paper sample corpus
(force-added to git; rest of data/ ignored)
    └── app/
        ├── __init__.py
        ├── main.py                     <- FastAPI entry point (/health,
/search, /metrics, /)
        ├── config.py                   <- pydantic-settings Settings;
builds database_url; LLM env vars
        ├── api/
        |   ├── __init__.py
        |   ├── routes_health.py        <- APIRouter with GET /health
        |   ├── routes_search.py        <- [M2] APIRouter with GET /search?
q=...&top_k=...
        |   └── routes_retrieve.py      <- [M3] APIRouter with GET /retrieve
(full hybrid pipeline)
        ├── db/
        |   ├── __init__.py
        |   ├── session.py              <- async engine, AsyncSessionLocal,
Base, get_db dependency
        |   ├── models.py               <- the 7 ORM tables
        |   └── qdrant_setup.py         <- creates the 'research_papers'
Qdrant collection (idempotent)
        └── services/
            ├── __init__.py
            ├── generation_client.py    <- swappable LLM client; stub mode +
OpenAI-compatible HTTP mode
            ├── ingestion.py            <- [M2] embed papers -> Qdrant
vectors + Postgres metadata (shared-UUID)
            ├── search.py               <- [M2] dense-only: embed query ->
Qdrant NN -> resolve via qdrant_id
            ├── bm25_retriever.py       <- [M3] in-memory BM25 sparse
retriever over Postgres chunks
            ├── fusion.py               <- [M3] reciprocal rank fusion (RRF,
k=60)
            ├── reranker.py             <- [M3] cross-encoder rerank (thesis
S2 ms-marco-MiniLM-L-6-v2)
            └── hybrid_search.py        <- [M3] orchestrator: BM25 + dense -
> RRF -> rerank -> results
```

Folders not yet created (added in later milestones, do not pre-create empty): `backend/app/workers/`, `backend/app/monitoring/`, `ml/`, `docs/`, `loadtest/`, `notebooks/`. (`backend/data/` now exists, created in M2.)

---

4. File-by-file: what each file does

- **`requirements.txt`** — pinned deps: `fastapi`, `uvicorn[standard]`, `sqlalchemy`, `asyncpg` (async Postgres driver), `qdrant-client`, `redis`, `pydantic-settings`, `prometheus-client`, `httpx` (async HTTP, used by the generation client), `sentence-transformers` (**[M2]** embedding model all-MiniLM-L6-v2; **[M3]** also the cross-encoder reranker), `rank-bm25` (**[M3]** sparse retrieval).
- **`.gitignore`** — ignores `__pycache__/`, `*.pyc`, `.env`, `.venv/`, `data/raw_pdfs/`, `*.db`, `.DS_Store`, and **[M2]** `backend/data/` (large corpora). Confirmed working (no pycache/secrets tracked). The sample corpus `backend/data/sample_papers.json` is force-added (`git add -f`) as the one tracked exception.
- **`.env.example`** — committed template documenting env vars. Updated to the real custom ports (see Section 5).
- **`docker-compose.yml`** — three services (postgres:15, qdrant/qdrant:latest, redis:7-alpine) with custom host port mappings and named volumes for Postgres and Qdrant. **No LLM container** (generation is an external HTTP service per the v2 plan).
- **`backend/app/config.py`** — `Settings(BaseSettings)`. Reads env vars (falls back to defaults). Exposes `settings.database_url` (a computed property assembling the async connection string) and LLM backend vars. One shared instance `settings`.
- **`backend/app/main.py`** — creates the FastAPI `app`, includes the health + search routers, mounts the Prometheus metrics app at `/metrics`, and defines a `/` root route.
- **`backend/app/api/routes_health.py`** — an `APIRouter` exposing `GET /health` → `{"status": "ok"}`.
- **`backend/app/api/routes_search.py`** — **[M2]** `APIRouter` exposing `GET /search?q=...&top_k=...`; validated query params (top_k 1–20); thin wrapper over `services/search.py` (dense-only).
- **`backend/app/api/routes_retrieve.py`** — **[M3]** `APIRouter` exposing `GET /retrieve?q=...&top_k=...&candidate_pool=...`; thin wrapper over `services/hybrid_search.py` (full BM25+dense+RRF+rerank pipeline). `/search` (dense baseline) and `/retrieve` (production retriever) both kept — useful for the M8 eval comparison.
- **`backend/app/db/session.py`** — `create_async_engine(settings.database_url)`, `AsyncSessionLocal` factory, `Base` (DeclarativeBase) that all models inherit, and the `get_db` FastAPI dependency that yields a session and auto-closes it.
- **`backend/app/db/models.py`** — the 7 ORM tables (see Section 6).
- **`backend/app/db/qdrant_setup.py`** — `ensure_collection()` creates `research_papers` (size 384, cosine) if missing; idempotent. Runnable via `python -m app.db.qdrant_setup`.

- `backend/create_tables.py` — imports models (registers them on `Base.metadata`), then runs `Base.metadata.create_all` against the live async engine. Run once to create tables.
- `backend/app/services/generation_client.py` — `GenerationClient.generate(prompt) -> str`. If `settings.llm_base_url == "stub"`, returns canned local text (no GPU/network). Otherwise POSTs to an OpenAI-compatible `/v1/chat/completions` endpoint and extracts `choices[0].message.content`. Shared instance `generation_client`. The HTTP path is written but only used from M4 onward.
- `backend/app/services/ingestion.py` — **[M2]** reads a papers JSON, and per paper: creates a `Paper` row (`flush()` to get `paper_id`), embeds the abstract (all-MiniLM, 384d), writes the vector to Qdrant under a `uuid4()`, writes a `Chunk` row with `qdrant_id` = that same UUID, then one `commit()` for the batch. Run: `python -m app.services.ingestion`.
- `backend/app/services/search.py` — **[M2]** dense-only search: embeds a query with the same model, runs Qdrant nearest-neighbor (`query_points`), resolves each hit to `Chunk`+`Paper` via `qdrant_id`. Run: `python -m app.services.search`.
- `backend/app/services/bm25_retriever.py` — **[M3]** sparse retriever. `BM25Retriever.build()` loads all chunk texts from Postgres and builds an in-memory `BM25Okapi` index; `.search(query, top_k)` returns ranked `(chunk_id, score)`. Shared `tokenize()` for docs and query. Run: `python -m app.services.bm25_retriever`.
- `backend/app/services/fusion.py` — **[M3]** `reciprocal_rank_fusion(ranked_lists, k=60, top_k)` — merges ranked chunk_id lists by `Σ 1/(k+rank)`, scale-invariant.
- `backend/app/services/reranker.py` — **[M3]** `rerank(query, candidates, top_k)` using `CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")` (thesis S2). Scores (query, doc) pairs jointly; returns ranked `(chunk_id, score)`. Scores are raw logits (ordering only).
- `backend/app/services/hybrid_search.py` — **[M3]** the full pipeline: dense leg (qdrant_ids→chunk_ids) + BM25 leg → RRF → fetch candidate texts → cross-encoder rerank → resolve final ids to text+provenance. `hybrid_search(query, candidate_pool=50, top_k=5)`. Run: `python -m app.services.hybrid_search`.
- `backend/data/sample_papers.json` — **[M2]** 6-paper sample corpus (RAG, hallucination, retrieval, faithfulness, reranking, calibration abstracts). Force-added to git; rest of `backend/data/` is gitignored.

---

5. CRITICAL: ports, credentials, and connection facts

These are **non-standard** because of conflicts on this Windows machine. Future-me must use these exact values.

Service	Container port	Host port (use this)	Why non-standard
Postgres	5432	**15432**	A native Windows Postgres service (postgres.exe, runs on boot) already owns host 5432.
Redis	6379	**16379**	Host 6379/6380 fell inside Windows reserved TCP port ranges.
Qdrant	6333	**16333**	Host 6333 fell inside a Windows reserved TCP port range (6271–6370).

> Inside the Docker network, services still use their normal internal ports (5432/6379/6333). The custom ports above are only for reaching the containers **from the laptop** (where the app currently runs via uvicorn).

Postgres credentials

- user = `research`, password = `research`, db = `research`
- App connection string (built by config): `postgresql+asyncpg://research:research@localhost:15432/research`

Qdrant

- Collection name: `research_papers`
- Vector size: **384** (all-MiniLM-L6-v2). Distance: **Cosine**.
- REST check: `curl http://localhost:16333/collections`

config.py defaults must match the above

- `postgres_port = 15432`, `redis_port = 16379`, `qdrant_port = 16333`
- LLM vars: `llm_base_url = "stub"`, `llm_api_key = "not-needed"`, `llm_model = "mistral-7b"`

---

6. Database schema (7 tables, as implemented)

All inherit from `Base`. Relationships (parent --< child): `papers --< chunks`, `research_jobs --< research_results`, `research_jobs --< claims`, `claims --< evidence`, `chunks --< evidence`, `research_jobs --< feedback`.

Table	Key columns	Purpose
`papers`	paper_id (PK), title, authors, year, source, pdf_path, created_at	Paper metadata
`chunks`	chunk_id (PK), paper_id (FK→papers), section, text, qdrant_id	Chunk text + the bridge to its Qdrant vector
`research_jobs`	job_id (PK), question, status, created_at, completed_at	One async research request (unit of work)
`research_results`	job_id (PK **and** FK→research_jobs), answer, grounding_score, unsupported_rate	Final answer + quality metrics (one-per-job enforced by PK=FK)
`claims`	claim_id (PK), job_id (FK), claim_text, support_score, label	Atomic claims + S2+S4 support score and Supported/Weak/Unsupported label
`evidence`	evidence_id (PK), claim_id (FK→claims), chunk_id (FK→chunks), evidence_text, source_title	Evidence linking a claim to the chunk that supports it
`feedback`	feedback_id (PK), job_id (FK), claim_id (FK, nullable), rating, comment, created_at	Optional user feedback on a job or a specific claim

`qdrant_id` on `chunks` is the link between Postgres (human metadata) and Qdrant (the embedding). This bridge gets used in M2.

---

7. How to bring the stack back up (resume checklist)

Every session, before working:

```
cmd
```

```
:: 1. Start Docker Desktop and wait until the whale icon is steady.
docker info

:: 2. From project root, start the data services:
cd C:\Users\mekal\verified-research-agent
docker-compose up -d
docker-compose ps          :: confirm postgres, qdrant, redis all "Up"

:: 3. Activate the right env and run the API from backend/:
conda activate fastapi-env
cd backend
uvicorn app.main:app --reload --port 8000
```

Sanity checks:

cmd

```
:: tables exist (run from project root)
docker-compose exec postgres psql -U research -d research -c "\dt"
:: qdrant collection exists
curl http://localhost:16333/collections
:: api health
:: open http://localhost:8000/health  -> {"status":"ok"}
:: generation stub round-trip (from backend/)
python -c "import asyncio; from app.services.generation_client import generatio
:: [M2] semantic search works (from backend/, with sample corpus ingested)
python -m app.services.search
:: [M2] search endpoint (server running): open
::   http://localhost:8000/search?q=detecting unfaithful generated text&top_k=3
```

Note: the 6-paper sample corpus persists in Docker volumes across restarts (Postgres + Qdrant both have named volumes), so you do **not** need to re-ingest each session. To re-ingest from scratch (e.g., after `docker-compose down -v`): `python -m app.services.ingestion` from `backend/`.

Two habits that prevent ~90% of confusing errors on this machine:

1. Before running anything, confirm the prompt shows `(fastapi-env)` and that you're in the `backend\` folder (imports like `from app...` only resolve from `backend/`).
2. When opening files with `code`, the path is relative to the current terminal directory — don't prefix with `backend\` when already inside `backend\`.

---

8. Gotchas already solved (so they aren't re-debugged)

(All from the M1 setup phase, but they apply whenever the stack is rebuilt.)

1. **Nested `backend\backend\` folder** — caused by running `code backend\app\main.py` while already inside `backend\`. Fix: paths are relative to the current dir.
2. **Windows reserved TCP port ranges** — `netsh interface ipv4 show excludedportrange protocol=tcp` lists them; 6333/6379 fell inside reserved ranges. Fix: use high host ports (16xxx).
3. **Postgres "password authentication failed" despite correct creds** — root cause was a **native Windows Postgres service (postgres.exe, PID was 7076) listening on host 5432**, intercepting connections meant for the container. Container env was correct the whole time. Fix: moved the container to host port **15432**. (Diagnosing it: `netstat -ano | findstr :5432` showed two listeners; `tasklist | findstr <pid>` revealed postgres.exe.)
4. **Postgres init-only credentials** — `POSTGRES_PASSWORD` is only applied on first init of an empty volume. To reset creds during dev: `docker-compose down -v` (the `-v` wipes the named volumes), then `up`. Confirm the volume is actually gone with `docker volume ls`.
5. **`.env` vs `.env.example`** — config reads a file named exactly `.env`. The committed template is `.env.example` (not read by config; defaults in config.py are what's used unless a real `.env` exists). A stray `.env.example.txt` duplicate was deleted.

---

9. Git / GitHub state

- Repo initialized; commits per milestone.
- **M1 commit (`220e305`):** `M1 complete: local stack (FastAPI, Postgres, Qdrant, Redis, Docker Compose) + generation stub` — 16 files, 454 insertions.
- **M2 commit (`1309425`):** `M2 complete: ingestion pipeline + semantic search + /search endpoint` — 7 files, 254 insertions.
- **M3 commit:** `M3 complete: hybrid retrieval (BM25 + dense + RRF + cross-encoder rerank) + /retrieve endpoint`.
- Remote: **https://github.com/Tharun2908/verified-research-agent** — pushed, `main` tracks `origin/main`.
- Per-milestone workflow going forward:

```cmd
git add -A
git commit -m "Mx complete: <short description>"
git push
```

- Harmless LF→CRLF warnings appear on Windows; ignore (optional fix: a `.gitattributes` with `* text=auto`).

---

10. The 12 milestones (v2 plan) and where we are

M#	Label	Scope	Status
M1	Setup	Repo, FastAPI, Postgres, Qdrant, Docker Compose, generation stub	**DONE**
M2	Ingestion	Embed (all-MiniLM, 384d) → vectors in Qdrant + metadata in Postgres (shared-UUID bridge) → semantic search + `/search` endpoint	**DONE**
M3	Retrieval	BM25 (rank-bm25) + dense (M2) + RRF fusion + cross-encoder rerank (thesis S2) → top evidence chunks; `/retrieve` endpoint	**DONE**
M4	Generation	Mistral-7B via vLLM OpenAI-compatible server on H200; backend calls it through the (already-built) client; swappable to API/stub by env var	**NEXT**
M5	Claim extraction	Split generated answer into atomic claims	
M6	Verification	Plug in thesis S2+S4 fusion verifier → per-claim support score + Supported/Weak/Unsupported label	
M7	Monitoring	Prometheus metrics + Grafana dashboards (esp. unsupported_claim_rate gauge). Do before M10.	
M8	Evaluation	Controlled comparison (Basic RAG vs RAG+citations vs Verified Agent) + mandatory LLM-as-judge arm; report unsupported-rate reduction + verifier↔judge agreement (κ)	
M9	Serving benchmark (H200)	vLLM throughput/latency/quantization (bf16 vs AWQ; FP8 if possible)/prefix-caching sweep → docs/serving.md	
M10	Load test + bottleneck fix	k6/locust drive full pipeline to saturation; find bottleneck via Grafana; one measured fix; before/after p95	
M11	Capacity & cost	queries/hour on one H200; €/1k verified queries self-hosted vs API; CPU-verifier vs API-judge cost	
M12	Public artifacts	Live demo (HF Spaces/Hetzner; verifier on CPU, generation via API), HF model uploads, blog post, hero README	

---

11. M6 Verifier — design decisions & open questions (READ before building M6)

This section captures design decisions made *after* M1, during planning discussion. It is the most important forward-looking part of this doc. The verifier step is one of Project 3's two differentiators (the other is measured serving engineering), so these decisions matter disproportionately.

11.1 What ships live vs. what is a measured benchmark

- **Live production default + public demo:** ship **S2+S4 lightweight verifier on CPU** (~61 ms/example, no GPU). This is the "hallucination check runs live for free" story and it holds **regardless of domain**. This is the load-bearing production path.
- **Cascade (S2+S4 → escalate the most-uncertain X% of claims → MiniCheck-7B on GPU):** implement as an **optional high-accuracy mode + a documented benchmark in `docs/`**, NOT as a load-bearing production feature. It runs on the cluster GPU, mirrors how the v2 plan treats H200 serving numbers (benchmark in repo, demo runs cheap elsewhere).
- **Never** put MiniCheck-7B in the live demo path: 659 ms median / 2501 ms p95 *per example* × N claims would blow the <1-minute demo target and force GPU into hosting, killing the CPU-only demo story.

11.2 The cascade only works in-domain — this is a known risk AND a feature

- Thesis result (already measured, see §12): on **RAGTruth (in-domain)** the cascade has a sweet spot — 30% escalation → F1 0.763 at 4× cost, beating both lightweight (0.710) and MiniCheck alone (0.726). On **HaluBench (out-of-domain)** there is **no sweet spot — monotonic improvement to 100% escalation**.
- **Mechanism:** the cascade escalates the claims the lightweight verifier is *most uncertain* about. Out-of-domain, the lightweight verifier is **miscalibrated**, so "escalate the uncertain ones" degenerates into "escalate ~everything" → the cascade collapses toward plain MiniCheck with extra latency. No efficiency win.
- **Consequence for P3:** the verifier was trained on RAGTruth; P3's corpus is scientific papers (PubMedQA/arXiv) = a domain shift. **Expect HaluBench-like behavior** (cascade may escalate near-everything, no sweet spot). This is the predicted, analyzable outcome — **report it as a finding, do not treat it as a failure.** ("A negative result you understand beats a fake positive.")

11.3 The narrative either way (measure first, then tell the true story)

- **If P3 corpus is closer to in-domain than feared:** sweet spot exists → clean efficiency story ("cascade matches MiniCheck quality at ~40% of its latency").

- **If P3 corpus is out-of-domain (likely):** "I deployed the cascade and measured that it provided no efficiency benefit on this domain — it escalated nearly everything because the lightweight tier wasn't calibrated for scientific text, consistent with my thesis HaluBench finding. The cascade's value is **conditional on calibration**; the right fix is to adapt the verifier in-domain, not to cascade harder." This demonstrates calibration-awareness and knowing the method's operating envelope — the safety-engineer signal, and the stated portfolio differentiator (mechanistic failure analysis).
- **Rule:** run the cascade on P3's actual corpus at M6/M8 and report the real curve. Do **not** pre-claim which regime it lands in (prep principle: never claim numbers before measuring on a defined eval set).

11.4 Engineering note — escalation must be batched/async from day one

- MiniCheck p95 = 2501 ms and batched throughput = 1.0 ex/s (throughput-bound by sub-claim explosion: ~7 sub-claims/example × ~7 tok/s). This is the **known M10 load-test bottleneck**, identified in advance.
- Design the escalation tier to **batch all escalated claims of a job and call MiniCheck async/batched**, never sequential per-claim. This pre-writes the M10 story: "I knew from thesis efficiency benchmarks that MiniCheck's tail latency would dominate under load, so I batched escalated claims — p95 dropped from A to B." (Anticipated bottleneck > discovered bottleneck.)

11.5 Verifier domain-adaptation — POST-M8 extension (optional, high value)

- **Idea:** fine-tune the verifier (specifically S4) to be in-domain for scientific papers so the cascade sweet-spot returns. Motivated directly by §11.2 (out-of-domain → cascade collapses → adapt verifier to restore it). Tight research arc; good blog-post material.
- **Scope it as a post-M8 extension, NOT MVP.** It is a research sub-project (labels, fine-tuning, eval, avoiding circularity), closer to a thesis chapter than an engineering milestone. P3's value does not depend on it — the honest "verifier is domain-bound" finding stands alone.
- **The hard problem = labels.** Training a faithfulness verifier needs claim↔evidence supported/unsupported labels. RAGTruth had 15,090 human-annotated; P3's corpus has none. Three label sources, in increasing risk:
  - **Option A (lowest risk):** use an existing in-domain labeled dataset (e.g., SciFact / SciTail / PubMedQA labels / biomedical NLI — **VERIFY what exists via search when you get there; do not trust memory, the dataset landscape shifts**). If one fits the claim↔evidence shape, fine-tuning S4 is a weekend.
  - **Option B (medium risk — distillation):** label corpus pairs with a strong model, fine-tune S4 on those labels. **CIRCULARITY TRAP:** do NOT label with MiniCheck and then compare against MiniCheck — the verifier can never beat its own teacher, and a sharp interviewer catches it instantly. If distilling, label with an *independent, stronger* judge (e.g., GPT/Claude-class) while MiniCheck stays the baseline, and disclose the distillation.

- **Option C (highest effort, highest credibility):** hand-label a few hundred pairs → a *gold* in-domain eval set. Valuable independently (M8 needs a gold eval set anyway). Often: hand-label gold eval (C) + fine-tune on (A or B) + report on the gold set.

- **First move when you reach this:** search/verify what in-domain labeled datasets exist now. Existing dataset → easy (Option A). None → it's the labeling problem (B/C).

11.6 Capacity/cost hook (M11)

- The cascade-vs-MiniCheck latency table (§12) feeds M11 directly: "my CPU verifier reaches quality X at Nx lower cost per claim than MiniCheck." Also note the thesis validated its cost model — the 11× parameter-count proxy ≈ 10.8× measured latency ratio. Mention that as a rigor signal ("cost model validated by measurement").

---

12. Key technical facts needed later (from thesis — do NOT re-derive)

These are needed at **M6 (Verification)**:

- **Verifier:** Logistic Regression fusion of **S2 + S4** with metadata. RAGTruth test F1 = 0.7099, AUROC = 0.8617, ECE = 0.105.
- **S2** = `cross-encoder/ms-marco-MiniLM-L-6-v2`, min aggregation over (answer-sentence × context-sentence) pairs, normalized. (This is also the M3 reranker.)
  - Normalization (exact, from RAGTruth train): `S2_MIN, S2_MAX = -11.430, 10.641`; `norm_s2 = max(0, min(1, (raw_min_relevance - S2_MIN)/(S2_MAX - S2_MIN)))`; hallucination prob = `1 - norm_s2`.
- **S4** = `cross-encoder/nli-deberta-v3-base` fine-tuned on RAGTruth train (15,090 examples), saved at `/workspace/signal4_model/` on the cluster.
  - **Loading is version-sensitive:** `transformers==4.44.0` pinned (newer breaks the load; vLLM installs may upgrade it — force back). Use `ignore_mismatched_sizes=True`. Input format: `answer [SEP] context`, `truncation=True`, `max_length=512`. Score direction: **higher = more hallucination** (no inversion).
- **Fusion features:** `[norm_s2_min, s4_score, task_type_onehot, model_onehot]`; fusion script `/workspace/fusion_logreg_s2s4.py`.
- **Label thresholds (mirror in UI colors):** 0.70–1.00 = Supported (green); 0.45–0.69 = Weak (amber); 0.00–0.44 = Unsupported (red).
- **Environment isolation rule:** the verifier service keeps its own pinned env (transformers 4.44.0); the vLLM serving stack keeps a modern env; they talk only over HTTP/files. Never break the thesis pod (it's in writing phase).

Cascade results (already measured in thesis — reuse, do NOT re-run):

RAGTruth (in-domain) — sweet spot at 30%:

Escalation	F1	Cost
0% (lightweight only)	0.7099	1.0×
10%	0.7293	2.0×
30% (BEST)	0.7628 (P 0.747 / R 0.779)	4.0×
50%	0.7653	6.0×
100% (MiniCheck only)	0.7260	11.0×

HaluBench (out-of-domain) — NO sweet spot, monotonic to 100%:

Escalation	F1	Cost
0%	0.3943	1.0×
30%	0.4718	4.0×
50%	0.5590	6.0×
100%	0.7205	11.0×

Measured efficiency (latency, batch=1 unless noted):

Component	Params	Disk (MB)	Median ms	p95 ms	Throughput	F1	AUROC
S2 (relevance)	22.7M	88	39	271	9.2 ex/s (seq)	0.630	0.723
S4 (DeBERTa ft)	184M	711	22	25	73.6 ex/s (b=32)	0.704	0.847
Fusion S2+S4	207M	—	61	—	~16 ex/s	0.710	0.862
MiniCheck-7B	7B	14,766	659	2501	1.0 ex/s (batched)	0.726	0.875
Cascade @30%	—	—	259	—	~3.9 ex/s	0.763	—

- **The verifier runs fine on CPU** — the public demo needs no GPU. **MiniCheck needs GPU** for any meaningful latency (hence: cascade benchmark = GPU/cluster; live verifier = CPU).

- Cost model validated: 11× parameter-count proxy ≈ 10.8× measured latency ratio.

---

13. Corpus status (data for ingestion)

- **M2 was built and proven on a small 6-paper sample set** (`backend/data/sample_papers.json`, in the repo). The ingestion + search pipeline is **data-independent** — feeding it the real corpus later is just pointing it at a different file (plus switching to batch embedding for speed).
- **The real corpus is NOT yet loaded, and the old `pubmedqa_chunks.json` file no longer exists** (it was a one-off artifact from earlier thesis work; not saved). To get a real corpus, regenerate it:
  - **PubMedQA** is public — pull it fresh from HuggingFace (`pubmed_qa`) or its GitHub release, then chunk a slice. This gives documented provenance for the README.
  - **arXiv abstracts** are an equally valid alternative (a small slice of ML/NLP abstracts).
- **When loading the real corpus:** switch `ingestion.py`'s per-chunk `model.encode()` loop to **batch** `model.encode([list_of_texts])` — far faster for thousands of chunks. (Flagged again in §14.)
- **Note on domain shift:** whatever corpus is chosen (PubMedQA/arXiv) is likely *out-of-domain* relative to the RAGTruth-trained verifier — this is expected and is the setup for the §11.2 cascade-calibration finding. Not a problem to fix at ingestion time; just be aware it propagates to M6.

---

14. Immediate next step (M4 — Generation)

M3 is DONE (hybrid retrieval, proven on the 6-paper sample set). What M3 built, for reference:

- `backend/app/services/bm25_retriever.py` — in-memory BM25 over Postgres chunks. Run: `python -m app.services.bm25_retriever`.
- `backend/app/services/fusion.py` — RRF (`Σ 1/(k+rank)`, k=60), scale-invariant fusion of ranked id-lists.
- `backend/app/services/reranker.py` — cross-encoder rerank using thesis S2 (`ms-marco-MiniLM-L-6-v2`).
- `backend/app/services/hybrid_search.py` — orchestrator: dense + BM25 → RRF → fetch texts → rerank → results. Run: `python -m app.services.hybrid_search`.
- `backend/app/api/routes_retrieve.py` — `GET /retrieve` (full pipeline). `/search` (dense baseline) kept too.

- **Funnel note:** on the 6-chunk corpus (`candidate_pool=50` = "everything", so the cost-saving funnel only *demonstrates correctness*, not speed. Speed benefit appears at scale (thousands → 50 → 10).

M4 (Generation) brings in the LLM. Per v2 plan, generation is an external vLLM HTTP service (OpenAI-compatible), called through the **already-built** (`generation_client.py`) (currently in stub mode). M4 work:

1. Build a **generator service** that takes a query + the top evidence chunks from (`hybrid_search`), builds a prompt (question + retrieved evidence, with citation markers), and calls (`generation_client.generate(...)`).
2. Develop entirely against **stub mode** (no GPU) — the stub returns canned text, proving the end-to-end flow (retrieve → build prompt → generate → cited answer) on the laptop.
3. The real **Mistral-7B via vLLM on H200** is a later config swap ((`LLM_BASE_URL` → port-forwarded vLLM server), used at M9–M11 for benchmarking. Stub/API mode is the dev default.
4. Expose an endpoint (e.g. (`/answer` or (`/research`)) that returns a draft cited answer over the retrieved evidence.

Deliverable: end-to-end cited answers (retrieve → generate) working against the stub; backend swappable to a real vLLM server or hosted API by env var. This is the last piece before M5 (claim extraction) and M6 (verification), where the answer gets decomposed and each claim scored.

Key reminder for M4 onward: generation env vars already exist ((`LLM_BASE_URL`, (`LLM_API_KEY`, (`LLM_MODEL`)) and the client already handles both stub and OpenAI-compatible HTTP paths. M4 is mostly prompt-building + wiring, not new infrastructure.

---

15. Interview talking points (code-anchored — defend because I built it)

Rule for this section: a point only goes here if it's tied to a real decision in the code, phrased as the question it answers. Pure facts (no code anchor) belong in the theory prep, not here. Add 1–2 per milestone at completion. Cross-references to the big-ticket stories live in §11–§12.

M1 — Setup

- **Config via environment variables ((`config.py` + pydantic-settings).** Decision: no hardcoded hosts; same code reads (`localhost`) locally and Docker service names in-container, switched by env var. → Answers *"how do you handle environment-specific configuration?"* and *"how does the same code run locally and in Docker?"*
- **Swappable generation client with a stub mode ((`generation_client.py`)).** Decision: one (`generate(prompt)->str`) interface; backend chosen by (`LLM_BASE_URL`) ("stub" = local fake, else OpenAI-compatible HTTP). → Answers *"how do you develop against*

an LLM you can't always run / have no GPU for?" and *"how would you make the generation backend swappable (self-hosted ↔ API)?"*

- **Two databases on purpose (Postgres + Qdrant).** Decision: Postgres for structured metadata queried by exact value; Qdrant for embeddings queried by semantic similarity — a relational DB can't do efficient nearest-neighbor. → Answers *"why two databases?"* / *"why not just one?"*

M2 — Ingestion & semantic search

- **Shared-UUID bridge between vector store and metadata store.** Decision: one `uuid4()` per chunk written to BOTH Qdrant (point id) and Postgres (`chunks.qdrant_id`); search resolves Qdrant hits back to real text/provenance through it. → Answers *"how do you keep your vector DB and metadata DB in sync?"* and *"a vector search returns an id — how do you get the actual content and its source?"*
- **`flush()` vs `commit()` in the ingestion transaction.** Decision: `flush()` the Paper to get the DB-assigned `paper_id` for the chunk's foreign key, then a single `commit()` for the whole batch (atomic; rolls back on mid-batch failure). → Answers *"walk me through your data layer"* / ORM transaction questions.
- **Same embedding model on query and chunks.** Decision: identical `all-MiniLM-L6-v2` in ingestion and search — embedding the query with a different model puts vectors in incompatible spaces and makes similarity meaningless. → Answers *"what's a subtle bug that silently breaks semantic search?"*
- **Vector size pinned to the model (384) at collection creation.** Decision: Qdrant collection sized 384 to match all-MiniLM output; a mismatch rejects inserts. → Answers *"how do the embedding model and vector DB have to agree?"*
- **Qdrant indexing threshold / brute-force below it.** Observation: `indexed_vectors_count=0` while `points_count=6` is expected — HNSW only builds past `indexing_threshold` (10k); below it Qdrant does exact brute-force, which is faster for tiny collections. → Answers *"how does your vector search scale?"* / *"does it build an index immediately?"*

M3 — Hybrid retrieval

- **Bi-encoder vs cross-encoder (dense search vs reranker).** Decision: dense retrieval uses a bi-encoder (query and docs encoded separately → doc vectors precomputed/stored in Qdrant → fast, scalable but misses fine-grained interaction); the reranker is a cross-encoder (query+doc encoded *together* → one relevance score → accurate but nothing precomputable, one forward pass per pair). → Answers *"bi-encoder vs cross-encoder?"* / *"why not just use the cross-encoder for everything?"*
- **The cost/accuracy retrieval funnel.** Decision: cheap retrievers (BM25 + dense) run over the whole corpus → ~50 candidates; expensive cross-encoder reranks only those 50 → final 10. Cross-encoder accuracy at a fraction of full-corpus cost. Same cheap-broad→expensive-narrow shape as the verifier cascade. → Answers *"how do you balance retrieval quality vs latency/cost?"*

- **RRF fuses by rank, not score (scale-invariance).** Decision: BM25 scores (~0–2.5) and cosine scores (~0–0.65) are on incomparable scales, so you can't average them; RRF (`Σ 1/(k+rank)`, k=60) uses only rank position, ignores raw scores, and rewards docs ranked high in *both* lists. → Answers *"how do you combine keyword and semantic retrieval?"* / *"why not just add the scores?"*
- **Why hybrid at all (BM25 + dense).** Decision: dense embeddings capture meaning but miss exact rare tokens (drug names, genes like BRCA1, acronyms, numbers); BM25 catches exact terms but is blind to paraphrase ("heart attack" ≠ "myocardial infarction"). Fusing covers both blind spots. → Answers *"why add keyword search if you already have embeddings?"*
- **BM25 is an in-memory index, not a DB.** Observation: `rank-bm25` builds the index at runtime from Postgres chunk texts (no persistent BM25 store), fine at this corpus size; at scale you'd back it with Elasticsearch/OpenSearch. Also: query and docs must use the *same* tokenizer (sparse echo of the same-embedding-model rule). → Answers *"how does your keyword search scale?"* / *"what breaks BM25 silently?"* (tokenizer mismatch)
- **Cross-encoder scores are raw logits, not probabilities.** Observation: reranker outputs ranged +7.5 to −11.4; only ordering is meaningful, not the absolute value (ties to thesis S2 normalization). → Answers *"can you interpret that relevance score as a confidence?"* (no, not without calibration)

Big-ticket stories (detail in §11–§12), listed here so they're not forgotten:

- Cascade verification is **calibration-conditional**: in-domain (RAGTruth) it beats the 7B model at ~40% of its latency; out-of-domain (HaluBench) it collapses to escalate-everything. → *"why not just use a big model to check claims?"* and demonstrates knowing a method's operating envelope.
- **MiniCheck tail latency (p95 2.5s) is the anticipated load bottleneck** → batch escalated claims; "anticipated > discovered."
- **Cost model validated by measurement** (11× param proxy ≈ 10.8× measured latency).
- **Verifier is domain-bound** — reported honestly, not hidden; motivates the post-M8 domain-adaptation extension.